Strategic and Engineering Playbook for the 42 CORE World Championship

To maximize your probability of winning the **42 CORE Game World Championship** on November 6–7, 2026[1], the evidence from competitive RTS postmortems and the CORE engine specifications points to a clear engineering conclusion: **you should build a deterministic, Rule-Based Hierarchical State Machine with Micro-Scoring Heuristics in C, and use your compute hardware to run automated multi-core parameter optimization (via Optuna or a Genetic Algorithm) rather than Reinforcement Learning (RL)**[3].

The apparent contradiction between the two pieces of advice you received comes down to the distinction between **deployment feasibility** and **competitive optimality**. ChatGPT is right about *deployment*: because the tournament runner compiles your repository inside an isolated container using make and executes ./bot without Python or an ONNX runtime[7], the most portable way to submit a machine learning model is a **Hybrid Architecture** where a small macro policy is trained in Python and exported as static C weight arrays (const float weights[]) in a header file (trained_policy.h). However, proving that a neural network *can* be compiled into C does not mean it is the strongest way to win. Even under ChatGPT's hybrid proposal, a neural network does not play the game by itself; you still must write the entire C micro-engine (pathfinding, deposit mining, gem pile pickup, transfer timing, and combat targeting) before the model has any valid macro actions to choose among[8].

With only 13 days remaining until the November 6 opening presentation[1], spending several days engineering a custom TCP/JSON Gymnasium wrapper and debugging RL policy convergence will starve your underlying C mechanics of development time. Moreover, having a local GPU or Google Colab will not meaningfully speed up RL training for 42 CORE, because the game server (server/) is a CPU-bound C++ program that steps ticks and communicates with bots via local network sockets[9]. A GPU accelerates large matrix multiplications inside deep neural networks, whereas during RL rollout collection your GPU will sit idle waiting for the CPU game server and socket I/O to process each tick. In contrast, using a multi-core CPU (or high-vCPU cloud instances) to run dozens of parallel headless matches that tune the heuristic weights of a deterministic C state machine translates every hour of compute directly into higher win rates[4].

1. Head-to-Head Comparison: Heuristics + Parameter Tuning vs. ChatGPT's Hybrid RL

Across grid-based real-time strategy and bot programming tournaments—such as *MIT Battlecode*, *Two Sigma Halite*, *CodinGame*, and *42 CORE*—postmortems from top finishers show that deterministic state machines paired with mathematical tile-scoring heuristics and evolutionary or Bayesian parameter optimization consistently outperform end-to-end and hybrid RL bots under tight competition deadlines[3]. While advanced competitors in certain turn-based *CodinGame* contests have occasionally reached top leagues using quantized

neural networks or Graph Neural Networks trained with PPO when backed by custom high-speed C++ simulators[14], multi-unit RTS tournaments with strict compilation rules and short development windows are dominated by hand-crafted micro heuristics and offline parameter tuning[5].

Deterministic State Machines With Tuned Parameters Maximize Win Probability Under a 13-Day Deadline

Engineering scorecard comparing bot architectures for the 42 Heilbronn CORE Game Championship (November 6–7, 2026)

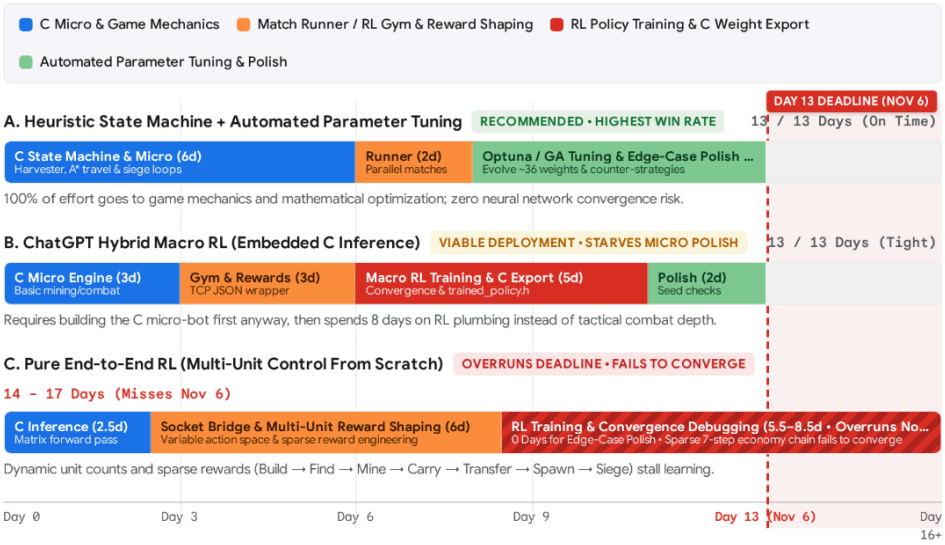
1. TOURNAMENT RUNNER CONTAINER LIMITS

Isolated Ubuntu Runner • Air-Gapped Loopback

COMPUTE LIMIT	MEMORY CAP	REPO VOLUME	BUILD & RUN
1 CPU Core	**512 MiB**	**250 MiB**	**make && ./bot**
Single-threaded C++ game server & bot execution; no GPU hardware	Strict per-bot RAM cap; heavy ML frameworks trigger OOM crash	Mounted writable at `/shared-data/repo`; zero internet access	Executes `cd my-core-bot && make && ./bot <rndId>` in C or Go

2. 13-DAY DEVELOPMENT TIME ALLOCATION (DAYS 0 TO 16 SCALE)

Championship Deadline: Day 13 (Nov 6)



3. SIMULATION MATCHES REQUIRED FOR CONVERGENCE (LOG10 SCALE) & RUNTIME PROFILE

Server Bottleneck: ~200–800 ticks/sec via TCP JSON

A. Heuristic State Machine + Optuna / Evolutionary Parameter Tuning

300 - 1,000 Matches

~2-6 hrs on multi-core CPU

Runtime Footprint: <0.1 ms/tick, <2 MB RAM

Micro Precision: Exact cooldown & damage math

Tournament Proof: Battlecode, Halite III & CodinGame winners

B. ChatGPT Hybrid Macro RL (Embedded C Weights in `trained_policy.h`)

5,000+ Matches

Days of local socket simulation

Runtime Footprint: <0.5 ms/tick (25-line C loop)

Macro Behavior: Probabilistic switches risk mistiming

Bottleneck: Requires completed C micro-engine before training

C. Pure End-to-End RL (PPO / SAC / MAPPO Raw Action Control)

50,000+ Matches (10M+ Ticks)

Weeks of CPU-bound JSON tick collection (GPU sits 99% idle)

Runtime Footprint: High risk of out-of-distribution stalls

Micro Precision: Fuzzy outputs lose 1v1 combat trades

Bottleneck: Single-threaded C++ server + JSON validation

100 Matches (10^2) | 1,000 Matches (10^3) | 10,000 Matches (10^4) | 100,000 Matches (10^5)

Comparison of development time allocation (across a 13-day window prior to the November 6 championship) and simulation sample requirements. While Hybrid C-Inference solves the container deployment blocker, it still requires building the full C micro-engine first and suffers from high sample complexity compared to evolutionary parameter tuning.

Data sources: CORE Game Wiki (Website Setup & Resource Limits) • Discover Heilbronn (CORE GAME 2026) • Battlecode 2025 Postmortem (Team SPAARK) • Battlecode 2024 Postmortem (Cout for Clout) • CodinGame Summer Challenge 2025 Postmortem • Halite III Bot Postmortem • Dota 2 AI State Machine Postmortem

To see why experienced tournament engineers favor heuristic state machines with automated parameter tuning over hybrid RL for 42 CORE, consider the five operational constraints summarized below:

Architectural Dimension	Pure End-to-End RL	ChatGPT's Hybrid RL (C Micro + Macro NN)	Heuristic State Machine + Genetic/Optuna Tuning
Container Compliance (1 CPU, 512 MiB RAM, 250 MiB Disk, make && ./bot)[7]	**Fails** if relying on Python/ONNX; requires custom C tensor math for variable unit arrays.	**Passes** by exporting small policy weights into a static C header (trained_policy.h).	**Passes natively** with minimal memory and CPU footprint using standard C structs[8].
Prerequisite C Coding Required	Minimal C wrapper, but infeasible to train multi-unit RTS actions from scratch in 13 days.	**Full C Micro required first** (mining, pathing, combat, transfers) before macro training can begin[8].	**Full C Micro + Macro**, written as modular, directly debuggable C functions[5].
Decision Precision & Edge Cases	Outputs stochastic action probabilities; prone to oscillating in corridors or wasting cooldown ticks.	Macro switches can jitter between ticks unless heavily smoothed and masked by hand-coded rules.	**Frame-perfect determinism**; evaluates exact integer HP, cooldowns, and Manhattan distances[18].
Compute Hardware Utilization	**CPU/Socket Bottleneck**; GPU waits on the single-threaded C++ game server per tick[9].	**CPU/Socket Bottleneck**; requires thousands of sequential socket episodes to stabilize policy gradients.	**Parallel CPU Scaling**; runs concurrent headless matches across all CPU cores to tune weights[4].
Adaptability on Tournament Day (Nov 6–7 Config	**High Risk**: If organizers modify unit costs or	**Fragile**: Feature distributions shift if game config	**Dynamic Adaptation**: Reads game.config at

Tweaks)[1]	deposit HP at kickoff, the network weights become invalid[21].	constants change mid-event[20].	runtime[21] and re-tunes parameters rapidly.

The "Config Change" Trap That Breaks Pre-Trained Neural Networks

A critical detail in the official CORE documentation (wiki/v0.1.0/documentation/configs) and release history makes pre-trained neural networks especially risky: **organizers regularly adjust config.game.json and unit component balance right before or during events**[20]. The official CORE changelog explicitly highlights releases such as *"Pre-event tweaks & config mid-event updating"* (v0.0.4.1) and *"Component-based gameplay refinements"* (v0.1.1.0), confirming that unit costs, deposit HP, deposit income, idle income, and component stats vary across events[20].

If you spend a week training a Hybrid RL policy on local configuration values—for instance, where a Deposit has 50 HP and yields 150 gems, Core HP is 200, and Initial Balance is 200[23]—and the tournament server launches on November 6 with updated component costs or deposit gem counts[1], your neural network's input normalization and learned thresholds will no longer match the game physics. Conversely, a deterministic C bot queries game.config and core_get_unitConfig() dynamically at runtime[11], adapting its affordability and cooldown math automatically to whatever configuration the tournament server loads[19].

Why Your GPU Won't Speed Up CORE—and How to Exploit Your Hardware Instead

In deep learning workloads, a GPU accelerates large tensor operations. However, in 42 CORE, even a hybrid macro neural network only takes a dozen scalar inputs (such as current gems, friendly miner count, visible enemy units, and elapsed ticks) and passes them through a tiny multi-layer perceptron. Computing that forward pass takes microseconds on a single CPU thread; transferring tiny vectors to a GPU over PCIe adds latency rather than removing it.

The actual training bottleneck is **match simulation**: running the C++ server binary, communicating over TCP sockets, and stepping through ticks[9]. Because the game server runs on the CPU, the effective way to leverage your hardware (or a multi-vCPU cloud environment) is **parallel match orchestration**[4]. As demonstrated by competitors in *Halite III*, *Battlecode*, and *CodinGame*, spawning 12 to 20 simultaneous headless matches across all logical CPU cores allows a Python script using **Optuna** or a **Genetic Algorithm** to evaluate hundreds of parameter combinations and converge on optimal strategic weights overnight[4].

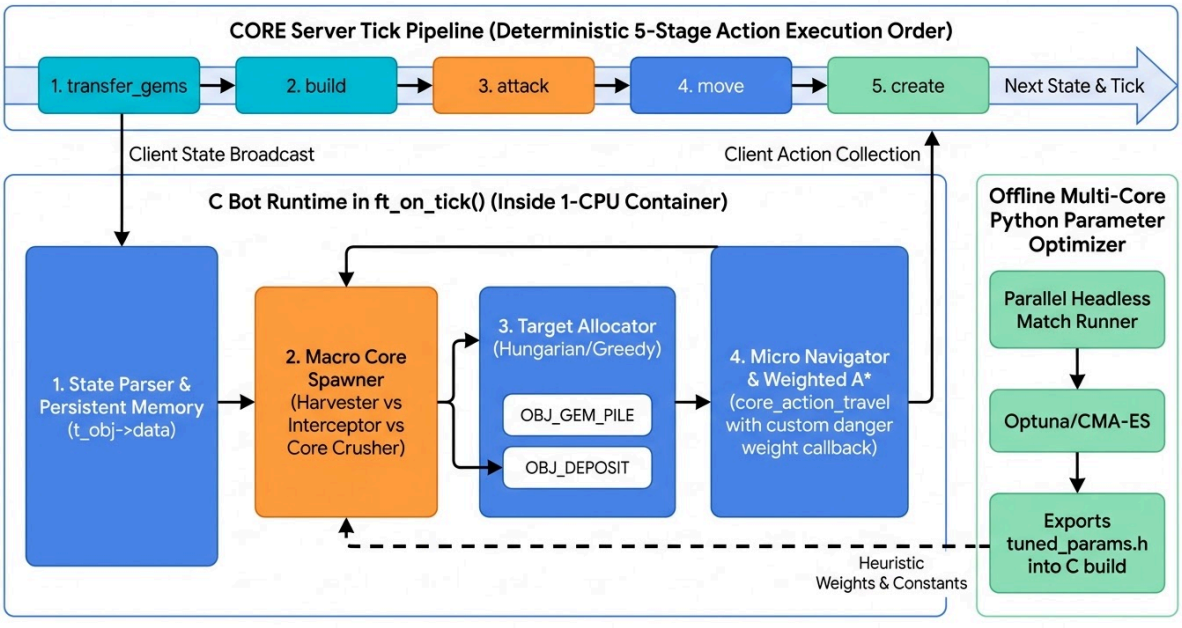
2. Tournament Server Architecture & Hard Constraints

Before implementing your strategy, your repository must strictly satisfy the deployment contract documented in Website Setup[7]. Any violation of these container constraints will prevent your bot from compiling or competing in the bracket[7]:

1. **Directory and Build Interface**: You must keep the my-core-bot folder name unchanged and ensure make produces an executable named ./bot inside that directory[7]. The tournament runner executes cd /shared-data/repo/my-core-bot && make && ./bot <rndId>[7], compiling everything inside my-core-bot/src/[24].
2. **Deployment Branch**: Only code pushed to the main branch of your assigned team repository is used for matchmaking queue games and the final tournament; commits on other branches are ignored[9].
3. **Filesystem and Network Isolation**: The container's root filesystem (/) is mounted read-only[7]. Your repository is mounted writable at /shared-data/repo with a volume size limit of 250Mi[7]. Network access for fetching external data is disabled, so any data or parameters must reside inside your repository and remain small[7].
4. **Compute and Memory Limits**: Typical per-bot container resource limits are CPU 1 and memory 512Mi[7]. In C, you must free every t_obj ** pointer array returned by library getter functions at the end of each tick (free(units)), while never freeing individual t_obj * pointers managed by the library[8].

3. Reverse-Engineering the CORE Engine Mechanics

Winning a deterministic grid strategy game requires designing your logic around the exact rules of the server. The CORE game takes place on a 2D square grid (game.grid_size) where at most one object (t_obj) can occupy a given (x, y) tile at any time, each object retains an unchanging unique id, and spatial calculations rely on **Taxicab / Manhattan distance** (|dx| + |dy|) rather than Euclidean distance[17]. Units can only attack or build on cardinally adjacent tiles—one tile up, down, left, or right[29].



1. Exploiting the Server's 5-Stage Action Execution Order

A crucial mechanic documented in Action Execution Order (wiki/v0.1.0/documentation/action_execution_order) governs how the server resolves commands sent by connected bots on the same tick. When ft_on_tick finishes, all actions from all connected clients are collected, sorted strictly by action type in the order transfer_gems → build → attack → move → create, and shuffled within each type[31].

Understanding this exact pipeline gives a rule-based C bot three distinct tactical advantages. First, because transfer_gems is applied *before* move and create, a miner standing adjacent to your Core at the start of a tick can call core_action_transferGems() to deposit its cargo, immediately issue a core_action_travel() command to step back toward a deposit in the move phase, and enable your Core to spend those newly deposited gems during the create phase—all within the exact same tick[8]. Second, because attack executes before move, if your combat unit is already cardinally adjacent to an enemy unit when its action_cooldown == 0[8], issuing an attack guarantees the damage lands before the target's move command can step it away[31]. Third, because multiple actions of the same type are shuffled randomly, asking a single unit to move to two different positions in the same tick produces random movement[31]; your state machine must enforce exactly one movement or attack decision per unit per tick and verify action_cooldown == 0 beforehand to keep action failures at zero[8].

2. The Exact Mathematics of Unit Action Cooldowns and Gem Weight

In 42 CORE, units cannot act every tick; performing an action sets the unit's action_cooldown, which decrements by 1 each tick until it reaches 0[8]. Crucially, carrying gems weighs a unit down and increases its cooldown after every step. According to the Cooldowns documentation (wiki/v0.1.0/documentation/cooldowns), a unit's action cooldown (ac) after an action is computed using integer division from its unit configuration (uconf = core_get_unitConfig(unit->s_unit.unit_type))[18]:

$$\mathrm{ac} = \min\left(\mathrm{maxActionCooldown},\ \max\left(1,\ \mathrm{baseActionCooldown} + \left\lfloor \frac{\max(\mathrm{gems}, 1)}{\mathrm{balancePerCooldownStep}} \right\rfloor\right)\right)$$

Exact Cooldown Step Mathematics and Directional Micro-Scoring Outperform Neural Approximations

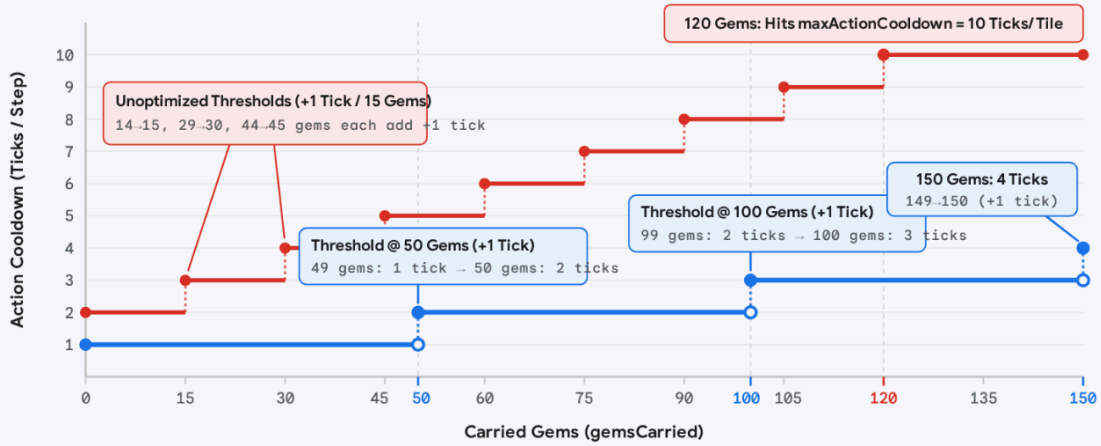
PANEL 1 — ENGINE PHYSICS & PAYLOAD SCALING

Unit Action Cooldown Penalty vs. Carried Gems

```
ac = min(maxAC, baseAC + ⌊gems / balancePerCooldownStep⌋)
```

Series A: Low Carry Efficiency Miner (baseAC=2, step=15, maxAC=10)

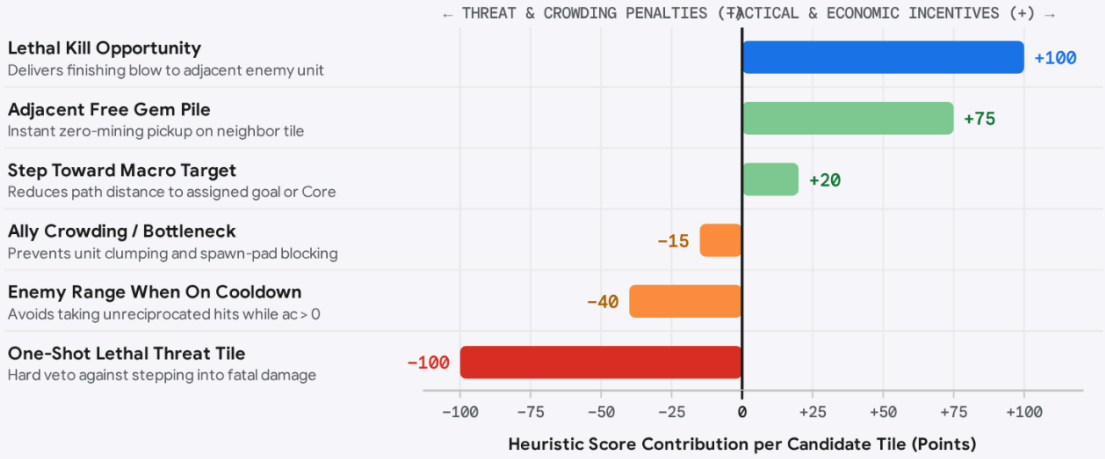
Series B: High Carry Efficiency Harvester (baseAC=1, step=50, maxAC=8)



PANEL 2 — XSQUARE / SPAARK TACTICAL MICRO ARCHITECTURE

Deterministic Tile-Scoring Heuristic Weights Across Candidate Moves (N, S, E, W, Stay)

```
d* = argmax_{d ∈ {N,S,E,W,Stay}} Σ w_i(tile_d)
```



Top: Unit action cooldown ticks required per step as a function of carried gems, governed by `baseActionCooldown` and `balancePerCooldownStep` (`gemsPerStep`). **Bottom:** Deterministic directional micro-scoring weights (adapted from championship Battlecode architectures) used to evaluate candidate movement tiles each tick.

Data sources: CORE Game Documentation, CORE Wiki: Cooldowns, Battlecode 2025 SPAARK Postmortem, Battlecode 2024 Cout for Clout Postmortem, Battlecode 2026 Postmortem.

Because C performs integer division (gems / uconf->balancePerCooldownStep)[18], the cooldown penalty increases in discrete steps rather than smoothly. To illustrate how this formula behaves, suppose a Harvester has baseActionCooldown = 2 and balancePerCooldownStep = 25 (Carry Efficiency / +gemsPerStep)[9]. When carrying 24 gems, its action cooldown is 2 + 24 / 25 = 2 ticks per step, meaning a 15-tile return trip to the Core takes 30 ticks. If that same miner picks up just 1 additional gem (25 gems total), its cooldown jumps to 2 + 25 / 25 = 3 ticks per step, increasing the 15-tile walk home to 45 ticks. By reading uconf->balancePerCooldownStep in C[18], your Harvester logic can cap its return threshold right below a step boundary (such as k * balancePerCooldownStep - 1) or evaluate the exact gems-per-tick return rate before deciding whether to mine another rock or head home[12].

4. The Championship Bot Architecture: 4 Core Subsystems

Drawing from the architectures of winning *MIT Battlecode* teams (*SPAARK*, *Cout for Clout*, *food*) and *Halite III* postmortems (*reCurs3*, *aidenbenner*)[5], your C codebase in my-core-bot/ should be organized into four modular subsystems.

Subsystem 1: Persistent Memory & Single-Pass Perception (state.c)

On every tick, the CORE client library updates the global game state (game.elapsed_ticks, game.my_team_id, game.grid_size, and the NULL-terminated game.objects array) before invoking ft_on_tick(unsigned long tick)[8]. Rather than treating every tick as stateless, you should exploit the **"Secret pro tip"** in the official CORE Tips and Tricks documentation: every t_obj struct contains a persistent void *data pointer[19]. Whenever a unit has unit->data == NULL, allocate a custom t_unit_brain struct and assign it to unit->data (remembering to free allocated memory cleanly at the end)[19]. Inside t_unit_brain, store the unit's current state (STATE_GATHER, STATE_RETURN, STATE_INTERCEPT, STATE_SIEGE), its locked target id, and its recent tile history so your movement scorer can penalize revisiting recent tiles and prevent pathing loops[5].

Subsystem 2: Unit Engineering & Macro Economy Manager (macro.c)

Since v0.1.0.0 (*"Component-driven units"*), players assemble custom unit roles using the event's Unit Builder and spawn them via core_action_createUnit()[8]. Each component modifies a unit's cost and stats across eight core properties: Unit Attack (+damageUnit), Core Attack (+damageCore), Mining (+damageObject), Armor (+damageReduction), Health (+hp), Speed (-actionCooldown), Gem Capacity (+maxGems), and Carry Efficiency (+gemsPerStep)[9]. As the My First Core Bot tutorial notes, an all-rounder unit that tries to do everything is slow and expensive, whereas splitting responsibilities into specialized, cheaper, and faster units is far more effective[8]. You can tag each role by passing its name as the first parameter to core_action_createUnit() and retrieve all living units of that role via core_get_units_byName()[8]:

Role Name (unit_name)	Component Stack Focus	Tactical Purpose	Spawn Trigger Condition (macro.c)
"Harvester"	Mining + Speed + Carry Efficiency + Gem Capacity[9]	Breaks OBJ_DEPOSIT rocks, collects dropped OBJ_GEM_PILE items, and returns gems to the Core with low cooldown penalties[8].	Spawn when own_core->s_core.spawn_cooldown == 0[8], gems are sufficient, and active Harvesters are below MAX_MINERS while deposits remain on the map.
"Interceptor"	Unit Attack + Speed + Health / Armor[9]	Prioritizes hunting enemy miners (collecting their dropped gems) and defending your Core and Harvesters from enemy fighters[8].	Spawn when harvester_count >= TARGET_EARLY_MINERS or immediately when an enemy unit approaches within DEFENSE_PANIC_RADIUS of your Core.
"Breaker"	Core Attack + Speed + Health + Armor (plus Mining if walls block paths)[8]	Bypasses skirmishes and focuses high +damageCore attacks directly on the opponent's Core (and OBJ_WALL obstacles if needed)[8].	Spawn once your economy is saturated (harvester_count >= MAX_MINERS) or when launching a coordinated squad push[13].

Subsystem 3: Bipartite Resource & Target Allocation (allocator.c)

In the starter tutorial, every miner searches for the closest deposit or gem pile independently using ft_get_deposit_gems_nearest(miner->pos)[8]. When you control multiple Harvesters, naive nearest-target selection causes all of them to walk toward the exact same resource tile; the first miner collects the gem pile while the others waste turns walking to an empty square[12].

To eliminate wasted movement, implement a centralized target assignment pass at the start of ft_on_tick (modeled on the bipartite matching and greedy cost-assignment algorithms used in *Halite III*)[12]:

1. Gather all resource objects (OBJ_GEM_PILE and OBJ_DEPOSIT) on the grid using core_get_objs_filter()[8].
2. For each gathering Harvester m_i and candidate resource r_i, evaluate a heuristic score estimating the net gems per tick:

   $$\text{Score}(m_i, r_j) = \text{BaseValue}(r_j) \cdot \text{PileMultiplier}(r_j) - w_{\text{dist}} \cdot \big(2 \cdot d(m_i, r_j) + d(r_j, \text{Core})\big) - w_{\text{threa}}$$

   Give OBJ_GEM_PILE objects a priority multiplier (WEIGHT_GEM_PILE_BONUS) because broken deposits drop gem piles on the map that require zero mining attacks to break—picking them up is immediate income[8].
3. Maintain a claimed_ids[] list during the tick loop (or solve the bipartite assignment matrix) so that once Harvester m_1 claims a gem pile or deposit, Harvester m_2 is automatically routed to the next-best unclaimed resource[12].

Subsystem 4: Weighted Pathfinding & Directional Micro Heuristics (micro.c)

While basic bots call core_action_travel(unit, target_pos, NULL) with NULL as the third argument[8], the CORE library documentation specifies that the final input to core_action_travel is a custom **weight function** callback that defines tile traversal costs during pathfinding[8]. By supplying a custom weight function, your Harvesters can assign elevated traversal costs to tiles adjacent to enemy combat units, automatically routing around danger zones while using the client library's built-in pathfinder[4].

For combat units (Interceptor and Breaker), combine macro concentration with local directional micro heuristics inspired by the *XSquare*, *Cout for Clout*, and *SPAARK* architectures in *Battlecode*[5]. At the macro level, apply **Lanchester's Square Law**: instead of trickling single fighters into outnumbered fights, rally newly spawned fighters into squadrons before pushing[13]. At the micro level, whenever a combat unit has action_cooldown == 0 and enemies are nearby, evaluate its immediate options deterministically: if an enemy unit or the enemy Core is already at Manhattan distance 1 (|dx| + |dy| == 1), issue core_action_attack() immediately so the blow lands in the attack phase before movement resolves[29]. Otherwise, score candidate movement targets by adding positive weights for closing distance to low-HP enemies (enabling focus-fire kills) and hunting enemy miners carrying gems (enemy->s_unit.gems > 0), while applying negative weights for stepping into outnumbered melee range when allies are not adjacent to support[5].

5. Production-Ready C Implementation Template

(my-core-bot)

The code below provides a clean, modular C foundation that enforces zero-error cooldown checks, simultaneous same-tick transfer_gems and movement, de-duplicated resource assignment, and strict memory cleanup[8].

1. Tunable Header File (my-core-bot/inc/tuned_params.h)

By isolating strategic thresholds and heuristic weights in a dedicated header file, your Python optimization script can rewrite these constants automatically before each local compilation[3]:

C

```
#ifndef TUNED_PARAMS_H
# define TUNED_PARAMS_H

// Macro Economy Parameters (Tuned via Offline Runner)
# define TARGET_EARLY_MINERS        2
# define MAX_MINERS                 4
# define MINER_CARRY_RETURN_THRESHOLD 45
# define DEFENSE_PANIC_RADIUS       6

// Resource Scoring Weights
# define WEIGHT_GEM_PILE_BONUS      1.65f
# define WEIGHT_DIST_PENALTY        1.20f

// Combat Target Priority Weights
# define SCORE_DEFENSE_URGENCY      500
# define SCORE_ENEMY_CARRYING_GEMS  150
# define SCORE_DIST_PENALTY         10
# define SCORE_HP_FOCUS_WEIGHT      2

#endif
```

2. Core Tick Loop & Harvester/Combat State Machine (my-core-bot/src/main.c)

C

```
#include "bot.h"
```

```
#include "tuned_params.h"
#include <stdlib.h>
#include <stdio.h>

typedef enum e_role_state {
   STATE_IDLE = 0,
   STATE_GATHER,
   STATE_RETURN
} t_role_state;

typedef struct s_unit_brain {
   t_role_state    state;
   unsigned long   target_id;
} t_unit_brain;

static int manhattan_dist(t_pos a, t_pos b) {
   int dx = (int)a.x - (int)b.x;
   int dy = (int)a.y - (int)b.y;
   if (dx < 0) dx = -dx;
   if (dy < 0) dy = -dy;
   return (dx + dy);
}

static bool is_resource(const t_obj *obj) {
   return (obj && (obj->type == OBJ_GEM_PILE || obj->type == OBJ_DEPOSIT));
}

static bool is_enemy_unit(const t_obj *obj) {
   return (obj && obj->type == OBJ_UNIT && obj->s_unit.team_id != game.my_team_id);
}

// Attach persistent state struct to unit->data
static t_unit_brain *get_brain(t_obj *unit) {
   if (!unit->data) {
      t_unit_brain *b = (t_unit_brain *)calloc(1, sizeof(t_unit_brain));
      unit->data = (void *)b;
   }
   return ((t_unit_brain *)unit->data);
}

// Harvester State Machine & De-duplicated Target Allocator
static void run_harvesters(t_obj *own_core) {
   t_obj **miners = core_get_units_byName("Harvester");
```

```
  t_obj **resources = core_get_objs_filter(is_resource);

  if (!miners) {
    if (resources) free(resources);
    return;
  }

  unsigned long claimed_ids[128];
  size_t claimed_count = 0;

  for (size_t i = 0; miners[i] != NULL; i++) {
    t_obj *m = miners[i];
    t_unit_brain *brain = get_brain(m);

    // 1. Server executes transfer_gems BEFORE move and create.
    // If adjacent to our Core with gems, transfer immediately AND allow same-tick movement!
    if (own_core && m->s_unit.gems > 0 && manhattan_dist(m->pos, own_core->pos) <= 1) {
      core_action_transferGems(m, own_core->pos, m->s_unit.gems);
      brain->state = STATE_GATHER;
    }

    // Guard against action failures: only move/attack when action_cooldown == 0
    if (m->s_unit.action_cooldown != 0)
      continue;

    // 2. State transition check
    if (m->s_unit.gems >= MINER_CARRY_RETURN_THRESHOLD || (!resources &&
m->s_unit.gems > 0)) {
      brain->state = STATE_RETURN;
    } else {
      brain->state = STATE_GATHER;
    }

    if (brain->state == STATE_RETURN && own_core) {
      core_action_travel(m, own_core->pos, NULL);
      continue;
    }

    // 3. Assign highest-scoring unclaimed deposit or gem pile
    t_obj *best_res = NULL;
    float best_score = -1e9f;

    for (size_t r = 0; resources && resources[r] != NULL; r++) {
```

```
        t_obj *res = resources[r];
        bool already_claimed = false;
        for (size_t c = 0; c < claimed_count; c++) {
          if (claimed_ids[c] == res->id) {
            already_claimed = true;
            break;
          }
        }
        if (already_claimed)
          continue;

        int d_to_res = manhattan_dist(m->pos, res->pos);
        int d_to_core = own_core ? manhattan_dist(res->pos, own_core->pos) : 10;
        float bonus = (res->type == OBJ_GEM_PILE) ? WEIGHT_GEM_PILE_BONUS : 1.0f;
        float score = (100.0f * bonus) - (WEIGHT_DIST_PENALTY * (float)(d_to_res * 2 +
d_to_core));

        if (score > best_score) {
          best_score = score;
          best_res = res;
        }
      }

      if (best_res) {
        if (claimed_count < 128)
          claimed_ids[claimed_count++] = best_res->id;
        core_action_travel(m, best_res->pos, NULL);
      } else if (m->s_unit.gems > 0 && own_core) {
        core_action_travel(m, own_core->pos, NULL);
      }
    }

    // Free NULL-terminated pointer arrays (never free individual t_obj* elements)
    free(miners);
    if (resources)
      free(resources);
}

// Combat State Machine: Core Defense, Miner Interception, and Core Siege
static void run_combat_units(t_obj *own_core, t_obj *enemy_core) {
    t_obj **fighters = core_get_units_byName("Interceptor");
    t_obj **enemies = core_get_objs_filter(is_enemy_unit);
```

```
    for (size_t i = 0; fighters && fighters[i] != NULL; i++) {
        t_obj *f = fighters[i];
        if (f->s_unit.action_cooldown != 0)
            continue;

        t_obj *best_target = NULL;
        int best_score = -999999;

        for (size_t e = 0; enemies && enemies[e] != NULL; e++) {
            t_obj *en = enemies[e];
            int d_me = manhattan_dist(f->pos, en->pos);
            int d_core = own_core ? manhattan_dist(own_core->pos, en->pos) : 99;

            int score = -(d_me * SCORE_DIST_PENALTY) - (int)(en->hp *
SCORE_HP_FOCUS_WEIGHT);
            if (d_core <= DEFENSE_PANIC_RADIUS)
                score += SCORE_DEFENSE_URGENCY;
            if (en->s_unit.gems > 0)
                score += SCORE_ENEMY_CARRYING_GEMS;

            if (score > best_score) {
                best_score = score;
                best_target = en;
            }
        }

        if (best_target)
            core_action_travel(f, best_target->pos, NULL);
        else if (enemy_core)
            core_action_travel(f, enemy_core->pos, NULL);
    }

    if (fighters) free(fighters);
    if (enemies) free(enemies);
}

void ft_on_tick(unsigned long tick) {
    t_obj *own_core = ft_get_core_own();
    t_obj *enemy_core = ft_get_core_opponent();

    // 1. Execute unit transfers and actions first
    run_harvesters(own_core);
    run_combat_units(own_core, enemy_core);
```

```
    // 2. Macro Spawner: verify spawn_cooldown == 0 and gem balance before creating units
    if (own_core && own_core->s_core.spawn_cooldown == 0) {
        size_t miner_count = core_get_units_byName_count("Harvester");
        t_obj **resources = core_get_objs_filter(is_resource);
        size_t res_count = 0;
        while (resources && resources[res_count])
            res_count++;
        if (resources)
            free(resources);

        // Replace component arguments below with exact strings/costs from your event's Unit
Builder
        int harvester_cost = 120;
        int interceptor_cost = 150;

        if (miner_count < TARGET_EARLY_MINERS || (miner_count < MAX_MINERS && res_count >
miner_count * 2)) {
            if (own_core->s_core.gems >= (unsigned long)harvester_cost)
                core_action_createUnit("Harvester", "mining", "speed", NULL);
        } else {
            if (own_core->s_core.gems >= (unsigned long)interceptor_cost)
                core_action_createUnit("Interceptor", "unit_attack", "speed", NULL);
        }
    }
    (void)tick;
}

int main(int argc, char **argv) {
    return (core_startGame("CORE_CHAMPION", argc, argv, ft_on_tick, false));
}
```

6. How to Use Your Compute Power: The Python Multi-Core Parameter Tuner

Instead of training an RL neural network, use your local multi-core CPU or cloud compute instances to optimize the constants in tuned_params.h via **Optuna** or a **Genetic Algorithm**[4]. In the *CodinGame Summer Challenge 2025*, top Legend League competitors tuned up to 36 evaluation parameters using Optuna self-play against earlier snapshots of their bots[6], while *Halite III* and *Battlecode* competitors used Python match runners (runmatches.py) to spawn parallel matches across all logical CPU cores and parse replay files for survival, resource

collection, and victory speed[4].
Follow this four-step procedure to build your automated tuning pipeline:

1. **Assemble an Opponent Pool**: Keep compiled binaries of the repository's built-in opponents (gridmaster and freddys-secret-bot) alongside historical snapshots of your own bot (bot_v1, bot_v2) so you do not overfit to a single opponent style[6].
2. **Generate Candidate Headers**: Have a Python script sample candidate parameter values (such as TARGET_EARLY_MINERS, MINER_CARRY_RETURN_THRESHOLD, WEIGHT_GEM_PILE_BONUS, and DEFENSE_PANIC_RADIUS), write them into my-core-bot/inc/tuned_params.h, and compile the C binary via make.
3. **Execute Parallel Headless Scrimmages**: Detect your machine's logical CPU cores and spawn concurrent matches across multiple map seeds[4].
4. **Evaluate Continuous Match Fitness**: Rather than scoring purely on binary win/loss, extract tick count, remaining Core HP, and total gems mined from the match logs or replay files[4] so that parameter sets achieving fast, decisive victories receive higher fitness scores than slow wins:

$$\text{Fitness} = \begin{cases} +1000 + (2000 - \text{elapsed_ticks}) + \text{own_core_hp} & \text{if Victory} \\ -1000 + \text{elapsed_ticks} - 5 \cdot \text{enemy_core_hp} & \text{if Defeat} \end{cases}$$

7. The 13-Day Execution Roadmap to November 6

The **CORE GAME World Championship** begins on Friday, November 6, 2026 (with the opening presentation at 17:45 and coding kickoff at 18:00) and culminates in the global tournament bracket on Saturday, November 7 at 19:00[1]. Execute the following five-phase plan over the next 13 days:

1. **Days 1–3 (Oct 24 – Oct 26): Devcontainer Setup & Zero-Error Micro Foundation** Spin up the local development environment (make devcontainer), inspect configs/ and core_print_config() to record exact component costs, deposit HP, and cooldown steps[11], and implement main.c with persistent unit->data pointers[19], strict action_cooldown == 0 / spawn_cooldown == 0 checks[8], and same-tick transfer_gems + move execution[31].
2. **Days 4–6 (Oct 27 – Oct 29): De-duplicated Resource Allocator & Custom Weighted A*** Implement the centralized resource allocator so multiple Harvesters never path to the same deposit or gem pile[12], and write a custom tile-weight callback function for core_action_travel() so miners route around enemy combat units and avoid narrow-corridor traffic jams[4].
3. **Days 7–9 (Oct 30 – Nov 1): Unit Builder Optimization & Combat Micro** Use the event Unit Builder to engineer specialized Harvester, Interceptor, and Breaker component combinations[8]. Add Lanchester squadron grouping, low-HP target focus, and enemy miner interception to your combat state machine[5].
4. **Days 10–11 (Nov 2 – Nov 3): Multi-Core Python Parameter Optimization** Connect your Python match runner and Optuna/Genetic tuner to tuned_params.h, run hundreds

of parallel matches overnight against gridmaster, freddys-secret-bot, and your own earlier checkpoints[4], and commit the highest-performing constants into your C source.

5. **Days 12–13 (Nov 4 – Nov 6): Online Queue Testing & Live Event Calibration** Push your verified C code to the main branch of your official event repository and test against live opponents in the website matchmaking Queue[9]. Immediately after the 18:00 kickoff on Friday, November 6[2], verify whether the championship config.game.json or Unit Builder parameters have been updated for the event[20], run a fast parameter tuning pass on the final competition config, and push your updated tuned_params.h to main ahead of Saturday's 19:00 tournament bracket[2].

Works cited

1. CORE GAME - Discover Heilbronn, https://heilbronn.app/events/core-game-evt20914f01b72254dd
2. CORE GAME World Championship Tickets, Multiple dates | Eventbrite, https://www.eventbrite.com/e/core-game-world-championship-tickets-2003296408485?aff=ebdssbdestsearch
3. Dota 2 Bot Development with Lua AI | PDF - Scribd, https://www.scribd.com/document/422560866/Scripting-Dota-2
4. Halite III bot post-mortem : r/GameDevelopment - Reddit, https://www.reddit.com/r/GameDevelopment/comments/1shv725/halite_iii_bot_postmortem/
5. https://battlecode.org/assets/files/postmortem-2025-spaark.pdf
6. Post Mortem - Developer Community Forum - CodinGame, https://www.codingame.com/forum/t/summer-challenge-2025-post-mortem/207074
7. Website Setup - CORE Game, https://coregame.sh/wiki/v0.0.3/documentation/website_setup
8. My First Core Bot, https://coregame.sh/wiki/my_first_core_bot
9. CORE Game, https://coregame.sh/
10. CORE game - GitHub, https://github.com/42core-team
11. function core_startGame(...) - CORE Game, https://coregame.sh/wiki/v0.1.0/reference/c/startGame
12. GitHub - aidenbenner/halite3: Code for the Halite 3 AI bot competition, https://github.com/aidenbenner/halite3
13. https://battlecode.org/assets/files/postmortem-2024-cout-for-clout.pdf
14. Summer Challenge 2024 - Developer Community Forum - CodinGame, https://www.codingame.com/forum/t/summer-challenge-2024-feedback-and-strategies/204471
15. Summer Challenge 2026 - Feedback & Strategies - CodinGame, https://www.codingame.com/forum/t/summer-challenge-2026-feedback-strategies/208574
16. https://alext.app/Battlecode_Postmortem_2026.pdf
17. CORE Game, https://coregame.sh/wiki/v0.1.1/reference/c/s_game

18. CORE Game, https://coregame.sh/wiki/v0.1.0/documentation/cooldowns
19. Tips and Tricks - CORE Game, https://coregame.sh/wiki/v0.1.0/tips_and_tricks
20. Changelog - CORE Game, https://coregame.sh/changelog
21. Configs - CORE Game, https://coregame.sh/wiki/v0.1.0/documentation/configs
22. Better resource functions · Issue #65 · 42core-team/connection, https://github.com/42core-team/connection/issues/65
23. CORE Game, https://coregame.sh/wiki/v0.1.0/reference/go/config/go_PrintConfig
24. 42core-team/my-core-bot: CORE Bot Repo Template - GitHub, https://github.com/42core-team/my-core-bot
25. the Official CORE Wiki, Documentation & Reference - CORE Game, https://coregame.sh/wiki/v0.0.5
26. CORE Game, https://coregame.sh/wiki/v0.0.4/
27. Wiki - CORE Game, https://coregame.sh/wiki
28. Basics - CORE Game, https://coregame.sh/wiki/v0.0.3/basics
29. CORE Game, https://coregame.sh/wiki/v0.1.0/reference/go/actions/Attack
30. CORE Game, https://coregame.sh/wiki/v0.1.0/reference/c/actions/core_action_build
31. Action Execution Order - CORE Game, https://coregame.sh/wiki/v0.1.0/documentation/action_execution_order
32. Halite AI Programming Competition - Wikipedia, https://en.wikipedia.org/wiki/Halite_AI_Programming_Competition
33. CORE Game, https://coregame.sh/wiki/v0.0.5/documentation/getter_filtering